

Inteligência Artificial | FGA0021 | Turma 01 | 2026.1

Professor: Dr. Fabiano Araujo Soares

Aluno: Bruno Cruz Garcia Rosa

Matrícula: 222006605

Aprendizado por Reforço

1. Explicação do Problema e Modelagem do MDP

O problema abordado neste projeto consiste no desenvolvimento e avaliação de um **agente autônomo** operando sob o paradigma do **Aprendizado por Reforço (RL)** para resolver um problema de navegação sequencial com restrições e tomada de decisão lógica. O ambiente é configurado como um mapa matricial discreto de dimensões 10x10 contendo obstáculos físicos (paredes), moedas, chave, porta trancada, armadilhas e um estado terminal de alta recompensa (tesouro).

O objetivo do agente é partir de uma coordenada fixa de origem **(1, 1)** e descobrir uma rota que maximize a recompensa líquida acumulada até alcançar o tesouro, respeitando o limite rígido de $MAX_STEPS = 200$. Para que o agente interaja matematicamente com o ambiente, o cenário foi formalizado como um **Processo de Decisão de Markov (MDP)** estruturado em cinco componentes principais :

- **Espaço de Estados (S)**: Definido como uma tupla tridimensional discreta compreendendo a posição bidimensional do agente e um marcador booleano de inventário: (Row, Col, Has_Key) . Essa modelagem representa dois estados matemáticos completamente distintos dependendo da posse da chave, alterando a matriz de transição do sistema.
- **Espaço de Ações (A)**: Composto por quatro ações discretas de movimentação ortogonal: $A = \{UP, DOWN, LEFT, RIGHT\}$.
- **Probabilidade de Transição $P(s'|s, a)$** : Determinística. O movimento pretendido ocorre integralmente, exceto quando colide com paredes ou com a porta trancada sem a posse da chave. Nesses cenários de colisão, o agente permanece retido na mesma coordenada de origem.
- **Função de Recompensa $R(s,a)$** : Cada movimentação regular gera um custo operacional fixo de -1. Colisões com paredes custam -10 pontos. O acionamento de uma armadilha dispara uma penalidade severa de -50. A coleta de moedas e chaves confere +30 e +35 pontos, respectivamente. Destrancar a porta atribui +50 pontos e o alcance do tesouro final concede +200 pontos.

2. Abordagem Metodológica e Aplicação dos Algoritmos

2.1. Fluxo de Engenharia e Treinamento do Agente

A aplicação prática da arquitetura de aprendizado foi orquestrada sequencialmente ao longo de **5.000 episódios** completos dentro da classe *Trainer*. A estrutura de dados para armazenamento do conhecimento foi implementada por meio de um dicionário dinâmico atuando como uma Tabela Q, onde as chaves representam o estado atual do agente e os valores internos mapeiam as utilidades das quatro ações possíveis: $Q(s, a) \in R$.

O pipeline operacional foi executado seguindo o algoritmo estruturado abaixo:

1. **Inicialização:** O ambiente é resetado para o estado inicial $(1, 1, False)$ e a tabela Q inicia inteiramente zerada.
2. **Seleção de Ações (ϵ -Greedy):** Para gerenciar o balanço entre exploração e uso do conhecimento, utilizou-se o método *choose_action*. O agente sorteia um valor aleatório uniforme. Se este for menor que a taxa de exploração atual (ϵ), o agente toma uma ação completamente ao acaso. Caso contrário, ele consulta a tabela Q e escolhe a ação de maior utilidade ponderada.
3. **Mecanismo de Desempate Justo:** Para evitar o viés determinístico comum do Python, foi implementado um filtro ativo que rastreia todas as ações que compartilham do mesmo valor máximo de Q e realiza um sorteio aleatório estrito exclusivamente entre elas, garantindo aleatoriedade real em estados inexplorados.
4. **Atualização Iterativa via Equação de Bellman:** A cada passo efetuado através do método *step*, o agente recebe a recompensa imediata e o próximo estado. O algoritmo calcula o valor máximo esperado para o estado futuro e atualiza o valor Q atual aplicando a **Equação de Bellman** com taxa de aprendizado $\alpha = 0.1$ e fator de desconto futuro $\gamma = 0.9$.
5. **Decaimento da Exploração:** Ao término de cada episódio, a taxa ϵ sofre um decaimento multiplicativo estável de 0.999, reduzindo gradualmente a aleatoriedade inicial ($\epsilon = 1$) até atingir o piso de estabilização de $\epsilon_{min} = 0.01$ nos episódios finais.

3. Resultados Obtidos e Análise das Métricas

Ao término do treinamento de **5.000 episódios**, os dados gerados pelo histórico do modelo foram compilados e plotados para análise visual e validação estatística.

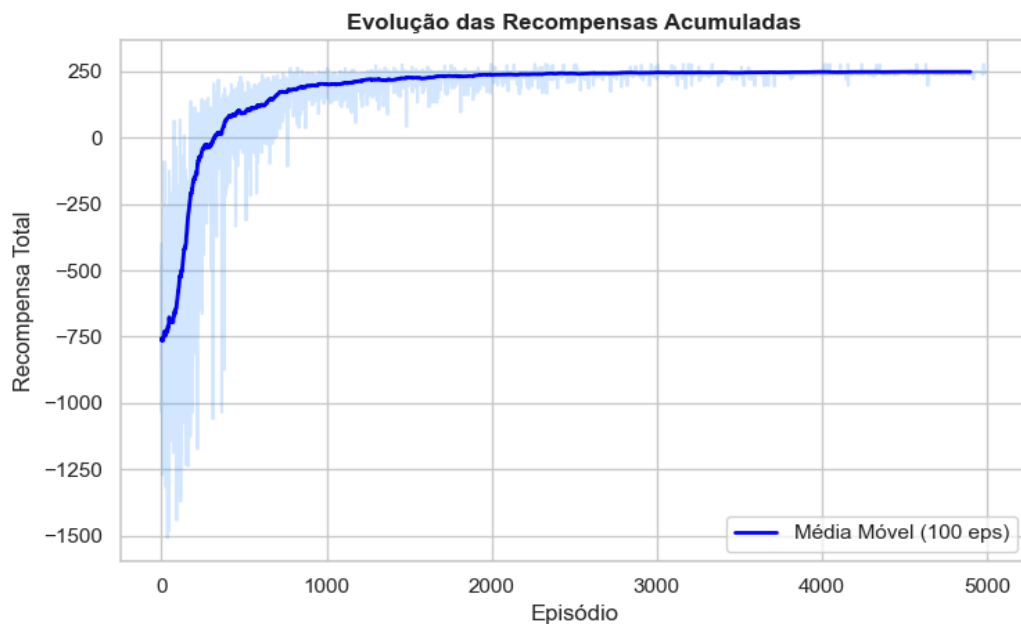


Figura 1 – Evolução das Recompensas Acumuladas ao longo dos episódios

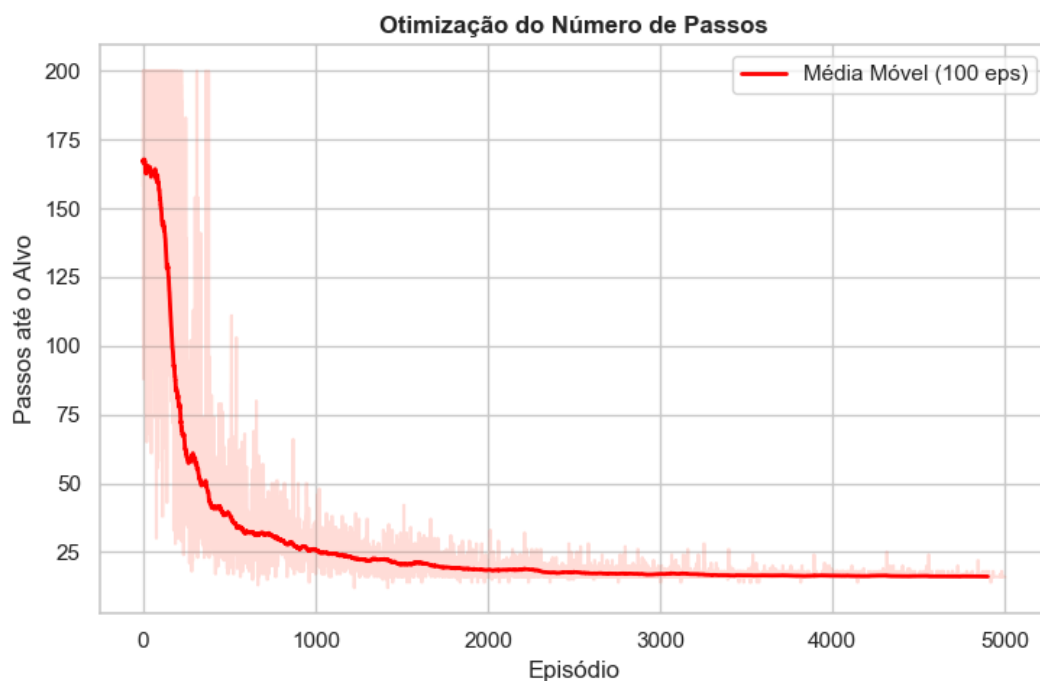


Figura 2 – Otimização do Número de Passos por Episódio

Para validar o comportamento definitivo do sistema, o agente foi submetido a uma rotina de avaliação determinística pura (*evaluate*), onde a taxa de exploração foi completamente desligada ($\epsilon=0$), forçando o robô a seguir rigorosamente a melhor política extraída de sua matriz de conhecimento. Os resultados numéricos finais sincronizados foram tabelados abaixo:

Tabela 1: Métricas Consolidadas da Rota de Avaliação Determinística

Métrica Avaliada	Valor obtido no teste
Recompensa Total Líquida	249 pontos
Passos Efetuados	16 Movimentos
Coleta de Recursos	1 Moeda e 1 Chave
Status Final de Inventário	<i>Has_Key = True</i>
Convergência Estatística	Estabilizada

O trajeto determinístico revelou uma estratégia puramente pragmática: o agente partiu de (1,1), coletou uma moeda em (3,2) e atravessou deliberadamente a armadilha em (3,3), absorvendo os -50 pontos de penalidade para acessar a zona interna e capturar a chave em (5,4). Contudo, portando o item (*Has_Key = True*), o algoritmo realizou um cálculo de utilidade inesperado: em vez de se mover até a porta trancada em (7,4), ele flanqueou o mapa utilizando a abertura da coluna 8. Dando a volta por fora da parede protetora, desceu de forma limpa pela lateral direita e alcançou o tesouro em (8,6) em apenas 16 passos, economizando movimentos e evitando riscos adicionais.

4. Considerações Finais e Limitações do Modelo

A análise analítica valida a eficácia prática do Q-Learning, mas expõe uma limitação de abordagens tabulares: a **convergência para um máximo local**. O agente descobriu uma rota funcional com retorno positivo estável. Contudo, sob a ótica de otimização global, esse caminho é subótimo. O mapa possui uma distribuição de moedas na bacia superior e uma dinâmica de chaves projetada para abrir uma porta avaliada em 50 pontos. Caso explorasse o labirinto de forma exaustiva, a recompensa líquida final superaria com folga a marca dos 300 pontos.

O motivo dessa estagnação reside no mecanismo **ϵ -Greedy** adotado. Como o atalho contornando a parede pela direita forneceu um ganho alto e rápido de 200 pontos nas fases intermediárias do treino, os valores Q dessa rota dispararam na tabela. Conforme a taxa de exploração decaiu para o piso de 0.01, o agente perdeu o ímpeto exploratório e passou a repetir de forma puramente gananciosa a primeira estratégia bem-sucedida encontrada.

Para que o robô quebrasse esse vício e descobrisse o máximo global absoluto, seriam necessárias modificações como a aplicação de uma **Inicialização Otimista na tabela Q** ou a calibração do *Reward Shaping* para penalizar a travessia de armadilhas de forma ainda mais proibitiva (ex: -300), inviabilizando matematicamente o atalho utilizado.